

- RevFit Recommender Engine Backend
 -  Directory Structure
 -  Core Engine Mechanisms
 - 1. Calorie & Macro Target Math
 - 2. Workout & Meal Scoring (Content-Based Filtering)
 - 3. Repetition Counter Integration
 -  API Routes Reference
 - Base URL
 -  User Profiles (/users)
 -  Workouts (/workouts)
 -  Meals (/meals)
 -  Daily Meal Plans (/meal-plan)
 -  Feedback Logger (/feedback)
 -  Pose Analysis (/pose)

RevFit Recommender Engine Backend

Welcome to the **RevFit Backend**, a FastAPI-powered recommendation engine and pose-analysis service for fitness and nutrition. This backend handles user profiling, calories/macros calculation, personalized workouts and meals recommendation (with feedback-loop scoring and time-decay), and computer vision-based exercise form validation.

Directory Structure

Here is an overview of the backend workspace structure:

- **[main.py](file:///home/kero/Github/Revfit/backend/main.py)**: The entry point of the FastAPI application. Includes middlewares, mounts static files, and aggregates the endpoints from all sub-routers.
- **[routers/](file:///home/kero/Github/Revfit/backend/routers)**: Sub-routers containing endpoints grouped by resource domains:
 - **[users.py](file:///home/kero/Github/Revfit/backend/routers/users.py)**: User profile management, calorie calculations, and session handling.
 - **[workouts.py](file:///home/kero/Github/Revfit/backend/routers/workouts.py)**: Catalog query, filter, and plan generators for workouts.
 - **[meals.py](file:///home/kero/Github/Revfit/backend/routers/meals.py)**: Recipe fetchers, hard filters, and scoring logic from Spoonacular.
 - **[meal_plan.py](file:///home/kero/Github/Revfit/backend/routers/meal_plan.py)**: Day-by-day daily meal plans with macro targets.
 - **[feedback.py](file:///home/kero/Github/Revfit/backend/routers/feedback.py)**: Like/dislike logging for both workouts and meals.
 - **[pose.py](file:///home/kero/Github/Revfit/backend/routers/pose.py)**: Video uploads for form scoring and correction.
- **[schemas.py](file:///home/kero/Github/Revfit/backend/schemas.py)**: Type-safe input/output models using Pydantic, enforcing constraints (e.g., matching fitness levels, diet types, goals).
- **[constants.py](file:///home/kero/Github/Revfit/backend/constants.py)**: Contains static configurations, workout splits (Push/Pull/Legs, Upper/Lower, Full Body), and decay definitions.
- **[state.py](file:///home/kero/Github/Revfit/backend/state.py)**: In-memory session store mapping active UUID tokens to user profiles and pose results.
- **[user_profile.py](file:///home/kero/Github/Revfit/backend/user_profile.py)**: Dataclasses for user details, including Mifflin-St Jeor BMR formulas, TDEE scaling, and feedback memory.
- **[filters.py](file:///home/kero/Github/Revfit/backend/filters.py)**: The core recommendation algorithm (hard constraint filtering + linear heuristic scoring + user feedback boosts).

- `[feedback.py]`(file:///home/kero/Github/Revfit/backend/feedback.py): Implements persistent storage (`feedback_store.json`) and **exponential decay** for workout preferences.
 - `[pose_analysis.py]`(file:///home/kero/Github/Revfit/backend/pose_analysis.py): Integrates with the `Pose/Code` module to run OpenCV-based headless inference on video feeds, drawing overlays and calculating rep counts/form scores. Uses `RepetitionCounter` for robust rep detection.
 - `[recommender.py]`(file:///home/kero/Github/Revfit/backend/recommender.py): Unified controller mapping workouts, meals, splits, and constraints into a single recommendation payload.
 - `[megaGymDataset.csv]`(file:///home/kero/Github/Revfit/backend/megaGymDataset.csv): Local workout dataset containing Title, Type, BodyPart, Equipment, Level, and Rating.
 - `[feedback_store.json]`(file:///home/kero/Github/Revfit/backend/feedback_store.json): Local persistence store for logged likes/dislikes.
-

Core Engine Mechanisms

1. Calorie & Macro Target Math

- **Basal Metabolic Rate (BMR)**: Computed using the Mifflin-St Jeor formula:
 - *Male*: $10 \times \text{weight (kg)} + 6.25 \times \text{height (cm)} - 5 \times \text{age (y)} + 5$
 - *Female*: $10 \times \text{weight (kg)} + 6.25 \times \text{height (cm)} - 5 \times \text{age (y)} - 161$
 - *Unspecified*: Average of male/female offsets.
- **Total Daily Energy Expenditure (TDEE)**: Scaled by activity levels:
 - `sedentary`: $\times 1.2$
 - `light`: $\times 1.375$
 - `moderate`: $\times 1.55$
 - `active`: $\times 1.725$
 - `very_active`: $\times 1.9$
- **Calorie Adjustments**: Applied based on goals (e.g., `fat_loss` subtraction of -500 kcal, `muscle_gain` addition of $+300$ kcal, capped at a minimum of 1200 kcal).
- **Macro Ratios**: Divided into target percentages (Protein / Carbs / Fat):
 - `fat_loss`: 40% / 30% / 30%
 - `muscle_gain`: 35% / 45% / 20%
 - `maintenance`: 30% / 40% / 30%
 - `endurance`: 25% / 55% / 20%

2. Workout & Meal Scoring (Content-Based Filtering)

Recommendations use a hybrid filter-score strategy:

1. **Hard Filters**: Eliminates items violating hard boundaries (e.g., equipment user doesn't have, level too advanced, allergens/intolerances present, prep time exceeding limits, calories exceeding meal capacity ceiling).
2. **Linear Preference Scoring**: Combines several factors:
 - *Goal Alignment*: Matching type to goals (e.g. Cardio heavily weighted for fat loss, Strength for muscle gain).
 - *Rating Bonus*: Minor score boost proportional to dataset rating.
 - *Feedback Memory Boost*: High weight multiplier applied to historically liked/disliked items.
3. **Preferences Decay**: Past likes/dislikes lose weight exponentially over time using:

$$\text{Decayed Score} = \sum (\text{delta}_i \times 0.5^{(\text{days_elapsed}_i / \text{half_life})})$$

Half-life values vary by goal (e.g., 14 days for fast-adapting `muscle_gain` training blocks, up to 45 days for `maintenance`).

3. Repetition Counter Integration

In the video feedback pipeline, the rep-counting logic has been refactored to use the `RepetitionCounter` module. Instead of manually inspecting phase state changes in a dictionary inside the loop, the system streams joint angles/coordinates to `RepetitionCounter.add_frame()` and queries `get_rep_count()` to fetch the updated count dynamically.

🔧 API Routes Reference

Base URL

By default, the server runs at: `http://localhost:8000`
Interactive Swagger docs: `http://localhost:8000/docs`

User Profiles (`/users`)

Manage user state, goals, preferences, and nutritional budgets.

HTTP Method	Route	Description	Request Payload	Response Model
POST	<code>/users/</code>	Registers a new session & profile. Computes target calories.	<code>UserCreateRequest</code> (body)	<code>{"session_id": str, "target_calories": int}</code>
GET	<code>/users/</code>	Lists all active session IDs stored in memory.	None	<code>{"sessions": List[str]}</code>
GET	<code>/users/{session_id}</code>	Retrieves the current user profile fields.	None	<code>UserResponse</code>
PUT	<code>/users/{session_id}/goal</code>	Updates user's goal type (e.g., fat loss) and updates calorie budget.	<code>UpdateGoalRequest</code> (body)	<code>UserResponse</code>
DELETE	<code>/users/{session_id}</code>	Destroys the user session.	None	<code>{"deleted": session_id}</code>
GET	<code>/users/{session_id}/macros</code>	Retrieves daily macro targets in grams based on calorie target.	None	<code>MacroTargetsResponse</code>

Workouts (/workouts)

Retrieve workouts and build structured split programs.

HTTP Method	Route	Description	Query Parameters	Request Payload / Response Model
GET	/workouts/	Paginated catalog queries.	<code>workout_type</code> (str), <code>body_part</code> (str), <code>equipment</code> (str), <code>level</code> (str), <code>limit</code> (int), <code>offset</code> (int)	Returns matching exercises: { <code>"total"</code> : int, <code>"workouts"</code> : List[WorkoutResponse]}
GET	/workouts/meta	Unique metadata filters (lists of body parts, levels, gear).	None	Unique strings for frontend dropdown selection
GET	/workouts/{workout_id}	Fetch a single workout details by database id.	None (pass <code>workout_id</code> in path)	WorkoutResponse
POST	/workouts/recommend/{session_id}	Predicts top-K workouts filtered & scored for this user.	<code>top_k</code> (int, default 5, range 1-50)	Request Body: None Response: List[WorkoutResponse]
GET	/workouts/filter/{session_id}	Hard-filtered exercises matching user's gear & level (unscored).	None	List[WorkoutResponse]
POST	/workouts/plan/{session_id}	Generates a structured multi-day training routine based on split settings.	<code>top_k</code> (int, default 5, range 1-50)	Request Body: None Response: WorkoutPlanResponse

Meals (/meals)

Integrates with Spoonacular to search and recommend recipes.

HTTP Method	Route	Description	Query Parameters	Request Payload / Response Model
POST	/meals/fetch/{session_id}	Fetches recipes matching user preferences with hard constraints (unscored).	num_fetch (int, default 20, range 5-100)	Request Body: None Response: {"fetched": int, "after_filter": int, "meals": List[RecipeResponse]}
POST	/meals/recommend/{session_id}	Generates top-K custom-tailored, ranked meals based on preferences.	top_k (int, default 5, range 1-50) num_fetch (int, default 20, range 5-100)	Request Body: None Response: List[RecipeResponse]
GET	/meals/recipe/{recipe_id}	Fetch a recipe directly from Spoonacular by ID.	None	RecipeResponse

Daily Meal Plans (/meal-plan)

Build daily nutrition charts structured into daily slots.

HTTP Method	Route	Description	Request Payload	Response Model
POST	/meal-plan/generate/{session_id}	Builds a daily menu (Breakfast, Lunch, Dinner, Snacks) with target matching.	MealRecommendRequest (body)	MealPlanResponse
GET	/meal-plan/slot-targets/{session_id}	Returns per-slot caloric/macro distribution targets.	None	Detailed breakdown based on meals-per-day

Feedback Logger (/feedback)

Provide implicit & explicit signals to fine-tune recommendation scores.

HTTP Method	Route	Description	Request Payload	Response
POST	/feedback/meal/{session_id}	Logs a like/dislike for a meal recipe.	MealFeedbackRequest (body)	{"status": "recipe_i

HTTP Method	Route	Description	Request Payload	Response
				"action":
POST	/feedback/workout/{session_id}	Logs a like/dislike for a workout.	WorkoutFeedbackRequest (body)	{"status": "workout_" "action":
GET	/feedback/{session_id}	Fetches recorded feedback & category preferences for a user.	None	Feedbacks
GET	/feedback/store/summary	Retrieves global database health statistics from feedback_store.json.	None	{"summary
DELETE	/feedback/{session_id}/reset	Clears all current session preferences in memory.	None	{"status": "session_

Pose Analysis (/pose)

Handle physical assessment of video uploads.

HTTP Method	Route	Description	Request payload (Multipart/Form)	Response Model
GET	/pose/exercises	List of exercise movements supported by the AI Pose module.	None	List[str]
POST	/pose/analyze/{session_id}	Upload a video file to receive rep counts, form scores, and skeleton corrections.	exercise_name: str (form) video: UploadFile (binary)	PoseAnalysisResponse (includes static url to correction video stream)
GET	/pose/history/{session_id}	Returns a session's history of all upload assessments.	None	List[PoseAnalysisResponse]

[!NOTE] Annotated form correction videos are stored inside backend/pose_outputs/ and exposed via the static mount /pose-outputs/.